

Manual Técnico – ESTFlix

Projeto

ESTFlix – Plataforma Web de Streaming de Filmes e Séries

Autor

David Fernandes – 2025141513

Diogo Castro – 2024121780

UC

Programação Web

Conteúdo

Resumo	4
1. Arquitetura do Sistema.....	5
1.1 Visão Geral	5
1.2 Estrutura de Diretórios.....	5
1.3 Módulos de Sistema.....	6
1.3.1 Módulo de Autenticação	6
1.3.2 Módulo de Gestão de Categorias.....	6
1.3.3 Módulo de Gestão de Conteúdos	6
1.4 Fluxo de Dados	7
2. Entidades e a sua Implementação	7
2.1 Descrição das Entidades	7
2.2.1 Utilizador (User)	7
2.2.2 Categoria (Category)	8
2.2.3 Conteúdo (Content)	8
2.2.4 Perfil (Profile).....	8
2.2.5 Favoritos (Favorites)	9
2.2.6 Histórico (History)	9
2.3 API REST	9
2.3.1 Rotas de Autenticação	9
2.3.2 Rotas de Categorias	10
2.3.3 Rotas de Conteúdos	10
2.3.4 Rotas de Perfis.....	10
3. Descrição das Opções Tomadas	11
3.1 Decisões Tecnológicas	11
3.1.1 Escolha do Backend: Node.js + Express	11
3.2 Decisões de Arquitetura	12
3.2.1 Separação de Responsabilidades	12
3.2.2 Middleware de Autenticação	12
3.2.3 Sistema de Recomendações	13
3.3 Decisões de Implementação.....	13
3.3.1 Gestão de Sessões	13
3.3.2 Validação de Dados	13
3.3.3 Tratamento de Erros	14
3.4 Decisões de Design	14
3.4.1 UI: Interface de Consumo vs Backoffice.....	14

3.4.2 Navegação SPA-like	14
4. Limitações Técnicas e Ideias para Desenvolvimento Futuro	15
4.1 Limitações Técnicas Atuais	15
4.1.1 Frontend	15
4.1.2 Backend	15
4.1.2 Base de Dados	16
4.2 Refactoring Necessário.....	16
4.2.1 Organização de Código	16
4.3 Ideias para Desenvolvimento Futuro	17
4.3.1 Novas Funcionalidades	17
4.3.2 Melhorias Técnicas	17
4.3.3 Melhorias de UX/UI.....	18
4.3.4 Expandir o Sistema.....	18
4.4 Conclusão.....	18
5. Esquema da Base de Dados	19
6. Open Source	20

Resumo

Este Manual Técnico documenta o desenvolvimento da aplicação web ESTFlix, uma plataforma de streaming inspirada em serviços modernos como a Netflix. O sistema foi desenvolvido no âmbito da unidade curricular de Programação Web, seguindo uma arquitetura cliente-servidor com frontend emHTML/CSS/JavaScript e backend em Node.js com Express e MySQL.

O projeto implementa funcionalidades completas de gestão de utilizadores, perfis, categorias e conteúdos, incluindo autenticação, favoritos, histórico de visualização e sistema de recomendações personalizadas. A aplicação segue os princípios REST para a API e utiliza programação orientada a objetos no frontend.

Este documento técnico destina-se a facilitar a compreensão do sistema por parte de técnicos que não participaram no desenvolvimento, documentando a arquitetura, entidades, decisões de implementação e limitações técnicas.

1. Arquitetura do Sistema

Este capítulo apresenta a arquitetura global do sistema ESTFlix, descrevendo a sua organização em módulos, as conexões entre os mesmos e o fluxo de informação dentro da aplicação.

1.1 Visão Geral

A aplicação ESTFlix segue uma arquitetura cliente-servidor com separação clara entre a camada de apresentação (frontend) e a camada de negócio (backend). O sistema é composto por dois componentes principais:

- Frontend: Implementado em HTML, CSS e JavaScript puro, sem recurso a frameworks como React ou Angular
- Backend: API REST desenvolvida em Node.js com Express

O frontend consome os serviços da API REST através de pedidos AJAX, permitindo uma experiência dinâmica sem recarregar a página.

1.2 Estrutura de Diretórios

A organização do projeto segue uma estrutura modular:

estflix/

src/

pages/ # Páginas HTML

scripts/ # Lógica JavaScript

models/ # Modelos de dados

services/ # Serviços e chamadas API

components/ # Componentes UI

pages/ # Controladores de página

styles/ # Folhas de estilo CSS

server.js # Ponto de entrada do servidor

database/

schema.sql # Esquema da base de dados

1.3 Módulos de Sistema

1.3.1 Módulo de Autenticação

O módulo de autenticação é responsável por gerir o acesso dos utilizadores ao sistema. Este módulo é implementado através das seguintes componentes:

- **Rotas API:** /api/auth/register, /api/auth/login, /api/auth/logout, /api/auth/me
- **Middleware:** Passport.js com LocalStrategy para autenticação baseada em email/password
- **Sessões:** express-session para gestão de estado de autenticação
- **Segurança:** bcrypt para hashing de passwords

Este módulo garante que todas as operações sensíveis requerem autenticação, sendo implementado um middleware de autenticação que protege as rotas dos restantes módulos

1.3.2 Módulo de Gestão de Categorias

O módulo de categorias permite a criação, leitura, atualização e eliminação de categorias de conteúdo. As categorias são usadas para organizar e filtrar os conteúdos disponíveis na plataforma.

- **Rotas API:** /api/categories (GET, POST), /api/categories/:id (GET, PUT, DELETE)
- **Validações:** Não é possível eliminar categorias com conteúdos associados
- **Requisitos:** Nome único por categoria

1.3.3 Módulo de Gestão de Conteúdos

O módulo de conteúdos é o núcleo do sistema, permitindo a gestão de filmes e séries disponíveis na plataforma.

- **Rotas API:** /api/contents (GET,POST),/api/contents/:id (GET,PUT, DELETE)
- **Filtros:** Por categoria, por termo de pesquisa (título ou descrição)
- **Validações:** Título único, ano entre 1900 e ano corrente + 1, classificação entre 0 e 5
- **Ordenação:** Por classificação (descendente) por defeito
- **1.3.4 Módulo de Perfis**
Cada utilizador pode ter múltiplos perfis, cada um com os seus próprios favoritos e histórico de visualização.
- **RotasAPI:** /api/profiles (GET,POST),/api/profiles/:id (GET,PUT, DELETE)
- **Favoritos:** /api/profiles/:id/favorites (GET,POST),/api/profiles/:id/favorites/:con (DELETE)

- Histórico: /api/profiles/:id/history (GET,POST),/api/profiles/:id/history/:content (DELETE)
- Recomendações: /api/profiles/:id/recommendations (GET) O sistema de recomendações analisa o histórico e os favoritos do perfil para sugerir conteúdos similares baseados nas categorias mais vistas.

1.4 Fluxo de Dados

O fluxo de informação no sistema ESTFlix pode ser descrito da seguinte forma:

1. O utilizador interage com a interface (ex: clicar em Adicionar aos favoritos)
2. O frontend envia um pedido AJAX para a API REST
3. O servidor recebe o pedido e valida a autenticação
4. O servidor processa o pedido e interage com a base de dados
5. A base de dados executa as operações CRUD necessárias
6. O servidor recebe os resultados e formata a resposta JSON
7. O frontend recebe a resposta e atualiza a UI sem recarregar a página

2. Entidades e a sua Implementação

Este capítulo descreve as entidades do sistema ESTFlix, os seus atributos, relacionamentos e a forma como são manipuladas através da API REST.

2.1 Descrição das Entidades

2.2.1 Utilizador (User)

A entidade Utilizador representa os utilizadores da plataforma. Cada utilizador tem uma conta com email e password, e pode ter múltiplos perfis associados.

Atributo	Tipo
id	INT AUTO_INCREMENT
email	VARCHAR(255)
Password hasheada com bcrypt	VARCHAR(255)
created_at	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)
updated_at	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)

2.2.2 Categoria (Category)

As categorias servem para classificar e organizar os conteúdos da plataforma.

Atributo	Tipo
id	INT AUTO_INCREMENT
name	VARCHAR(100)
color	VARCHAR(20) (DEFAULT '#e50914')

2.2.3 Conteúdo (Content)

Os conteúdos representam os filmes e séries disponíveis na plataforma.

Atributo	Tipo
id	INT AUTO_INCREMENT
title	VARCHAR(255)
description	TEXT
category_id	INT
year	INT
rating	DECIMAL(3,1) (DEFAULT 0)
image_url	VARCHAR(500)
created_at	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)
FOREIGN KEY (user_id)	REFERENCES users(id) ON DELETE CASCADE

2.2.4 Perfil (Profile)

Os perfis permitem que um utilizador tenha múltiplas identidades dentro da plataforma, cada uma com os seus próprios favoritos e histórico.

Atributo	Tipo
id	INT AUTO_INCREMENT
user_id	INT
name	VARCHAR(100)
avatar	VARCHAR(10)
created_at	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)
FOREIGN KEY (user_id)	REFERENCES users(id) ON DELETE CASCADE

2.2.5 Favoritos (Favorites)

Tabela de associação muitos-para-muitos entre perfis e conteúdos, que regista quais os conteúdos favoritos de cada perfil.

Atributo	Tipo
id	INT AUTO_INCREMENT
profile_id	INT
content_id	INT
created_at	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)
FOREIGN KEY (profile_id)	REFERENCES profiles(id) ON DELETE CASCADE
FOREIGN KEY (content_id)	REFERENCES contents(id) ON DELETE CASCADE
UNIQUE KEY	(profile_id, content_id)

2.2.6 Histórico (History)

Atributo	Tipo
id	INT AUTO_INCREMENT
profile_id	INT
content_id	INT
watched_at	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)
FOREIGN KEY (profile_id)	REFERENCES profiles(id) ON DELETE CASCADE
FOREIGN KEY (content_id)	REFERENCES contents(id) ON DELETE CASCADE

2.3 API REST

2.3.1 Rotas de Autenticação

Método	Rota	Descrição
GET	/api/auth/me	Obter utilizador autenticado POST
POST	/api/auth/register	Registar novo utilizador
POST	/api/auth/login	Iniciar Sessão
GET	/api/auth/logout	Terminar Sessão

2.3.2 Rotas de Categorias

Método	Rota	Descrição
GET	/api/categories	Listar todas as categorias
GET	/api/categories/:id	Obter categoria por ID
POST	/api/categories	Criar nova categoria
PUT	/api/categories/:id	Atualizar categoria
DELETE	/api/categories/:id	Remover categoria

2.3.3 Rotas de Conteúdos

Método	Rota	Descrição
GET	/api/contents	Listar todos os conteúdos (com filtros)
GET	/api/contents/:id	Obter conteúdo por ID
POST	/api/contents	Criar novo conteúdo
PUT	/api/contents/:id	Atualizar conteúdo
DELETE	/api/categories/:id	Remover conteúdo

2.3.4 Rotas de Perfis

Método	Rota	Descrição
GET	/api/profiles	Listar perfis do utilizador
GET	/api/profiles/:id	Obter perfil por ID
POST	/api/profiles	Criar novo perfil
PUT	/api/profiles/:id	Atualizar perfil
DELETE	/api/profiles/:id	Remover perfil
GET	/api/profiles/:id/favorites	Listar favoritos do perfil
POST	/api/profiles/:id/favorites	Adicionar aos favoritos
DELETE	/api/profiles/:id/favorites/:contentId	Remover dos favoritos
GET	/api/profiles/:id/history	Listar histórico do perfil
POST	/api/profiles/:id/history	Adicionar ao histórico
DELETE	/api/profiles/:id/history/:contentId	Remover do histórico
GET	/api/profiles/:id/recommendations	Obter recomendações

3. Descrição das Opções Tomadas

Este capítulo documenta as principais decisões de implementação tomadas durante o desenvolvimento do projeto ESTFlix, explicando os motivos que levaram à escolha de cada abordagem.

3.1 Decisões Tecnológicas

3.1.1 Escolha do Backend: Node.js + Express

Optou-se por utilizar Node.js como runtime e Express como framework web pelas seguintes razões:

- **Simplicidade:** Express é um framework minimalista que permite criar APIs REST de forma rápida e eficiente
- **JavaScript Full-Stack:** Permite utilizar JavaScript tanto no frontend como no backend, facilitando a manutenção e reduzindo a curva de aprendizagem
- **Comunidade e Ecossistema:** Node.js tem uma comunidade ativa e um vasto ecossistema de pacotes npm
- **Performance:** Node.js é adequado para aplicações I/O-bound como APIs REST

3.1.2 Base de Dados: MySQL

A escolha do MySQL como sistema de gestão de base de dados justifica-se por:

- **Relacional:** O modelo de dados do ESTFlix beneficia de relacionamentos bem definidos entre entidades
- **Maturidade:** MySQL é um SGBD maduro e amplamente utilizado na indústria
- **Integração:** Boa integração com Node.js através de bibliotecas como mysql2
- **Requisitos do Projeto:** O enunciado da UC pedia explicitamente a utilização de MySQL

3.1.3 Frontend: Vanilla JavaScript

Decidiu-se implementar o frontend utilizando HTML, CSS e JavaScript puro, sem frameworks, por várias razões:

- **Requisitos da UC:** O enunciado do projeto especificava explicitamente para não utilizar frameworks como React ou Angular
- **Aprendizagem:** Permite um melhor entendimento dos conceitos fundamentais de manipulação do DOM e programação orientada a objetos em JavaScript
- **Simplicidade:** Para um projeto de dimensão reduzida, vanilla JS é suficiente e evita a complexidade desnecessária de frameworks
- **Performance:** Sem a overhead de frameworks, o resultado final é mais leve

3.2 Decisões de Arquitetura

3.2.1 Separação de Responsabilidades

Optou-se por uma arquitetura em camadas com separação clara de responsabilidades:

- **Models:** Responsáveis pela representação dos dados e lógica de negócio
- **Services:** Responsáveis pela comunicação com a API e transformação de dados
- **Components:** Responsáveis pela apresentação e interação com o utilizador
- **Pages:** Responsáveis pela orquestração dos componentes e fluxo da aplicação

Esta separação melhora a manutenibilidade do código e permite que cada parte do sistema possa ser testada e modificada independentemente.

3.2.2 Middleware de Autenticação

Implementou-se um middleware de autenticação global que protege todas as rotas sensíveis. Esta abordagem tem as seguintes vantagens:

- **Centralização:** A lógica de autenticação está num só lugar, evitando duplicação de código
- **Manutenibilidade:** Qualquer alteração na lógica de autenticação apenas precisa de ser feita num ficheiro
- **Segurança:** Garante que nenhuma rota sensível é esquecida sem proteção
O middleware utiliza Passport.js com LocalStrategy, que fornece uma solução robusta e bem testada para autenticação baseada em sessões.

3.2.3 Sistema de Recomendações

O algoritmo de recomendações implementado segue uma abordagem baseada em conteúdo (content-based filtering):

1. Identificar as 3 categorias mais vistas/favoritadas pelo utilizador
2. Excluir conteúdos já vistos ou favoritos
3. Retornar os conteúdos melhor pontuados nas categorias preferidas
4. Se não houver resultados, fell back para conteúdo aleatório não visto

Esta abordagem foi escolhida porque:

- **Simplicidade:** É fácil de implementar e entender
- **Eficiência:** Não requer cálculos complexos ou machine learning
- **Efetividade:** Para um sistema com um catálogo não muito extenso, funciona bem

Alternativas consideradas: Collaborative filtering, que requer mais dados e é mais complexo de implementar.

3.3 Decisões de Implementação

3.3.1 Gestão de Sessões

Optou-se por utilizar express-session com cookies para gestão de sessões. Esta decisão baseou-se em:

- **Simplicidade:** Integração fácil com Express e Passport
- **Estado:** Permite manter estado entre pedidos HTTP (stateless por defeito)
- **Segurança:** Cookies podem ser configurados como HttpOnly e Secure

Os cookies são configurados com um tempo de vida de 7 dias, permitindo que os utilizadores permaneçam autenticados durante várias sessões.

3.3.2 Validação de Dados

Todas as validações são feitas no servidor, nunca apenas no cliente. Esta decisão é crítica por:

- **Segurança:** Validação no cliente pode ser contornada
- **Consistência:** Garante que todos os dados na base de dados estão válidos
- **Manutenibilidade:** Centraliza as regras de negócio num só lugar

As validações incluem:

- Campos obrigatórios
- Restrições de unicidade (emails, títulos, nomes de categorias)

- Restrições de chaves estrangeiras
- Valores dentro de intervalos (ano, classificação)

3.3.3 Tratamento de Erros

Implementou-se um tratamento de erros consistente em todas as rotas da API:

- **Códigos HTTP:** Utilização adequada de códigos de estado HTTP (200, 201, 400, 401, 403, 404, 409, 500)
- **Mensagens de Erro:** Mensagens claras e informativas para o cliente
- **Logging:** Erros são registados na consola para debug

Esta abordagem facilita o desenvolvimento do frontend e a manutenção da API.

3.4 Decisões de Design

3.4.1 UI: Interface de Consumo vs Backoffice

O projeto distingue claramente dois tipos de interfaces:

- **Interface de Consumo:** Páginas com cartões visuais para navegação de conteúdos (estilo Netflix)
- **Interface de Gestão:** Tabelas com operações CRUD para administração de dados (estilo backoffice) Esta distinção foi feita porque:
 - **Usabilidade:** Cada tipo de interface é otimizado para o seu propósito
 - **Clareza:** Utilizadores não são confundidos com interfaces inadequadas
 - **Requisitos:** O enunciado do projeto pedia explicitamente esta separação

3.4.2 Navegação SPA-like

Implementou-se uma navegação estilo Single Page Application (SPA) utilizando:

- **AJAX:** Para carregar dados sem recarregar a página
- **History API:** Para atualizar a URL sem recarregar a página
- **Event Delegation:** Para lidar com eventos em elementos dinâmicos

Isto proporciona uma experiência de utilizador mais fluida e moderna, embora sem a complexidade de um framework SPA completo.

4. Limitações Técnicas e Ideias para Desenvolvimento Futuro

Este capítulo identifica as principais limitações técnicas do sistema atual e propõe sugestões para melhorias futuras que não foi possível implementar devido a restrições de tempo ou complexidade.

4.1 Limitações Técnicas Atuais

4.1.1 Frontend

- **Falta de Framework Moderno:** O uso de vanilla JavaScript, embora educativo, limita a escalabilidade e manutenibilidade do código. Uma framework como React ou Vue.js tornaria o desenvolvimento mais rápido e o código mais organizado.
- **Estilos CSS Simples:** Os estilos atuais são básicos e não utilizam preprocessadores como SASS ou Tailwind CSS, o que torna o CSS mais difícil de manter em projetos maiores.
- **Acessibilidade:** Não foram implementadas todas as boas práticas de acessibilidade (ARIA labels, keyboard navigation, etc.).
- **Testes Unitários:** Não há testes automatizados para o frontend, o que dificulta a detecção de regressões.

4.1.2 Backend

- **Falta de Middleware de Validação:** As validações são feitas manualmente em cada rota, o que leva a duplicação de código. Um middleware de validação ou uma biblioteca como Joi ou express-validator seria útil.
- **Tratamento de Erros Centralizado:** Atualmente, o tratamento de erros é feito individualmente em cada rota. Um error handler centralizado melhoraria a consistência e manutenibilidade.
- **Logging Avançado:** O logging atual é básico (console.error). Um sistema de logging mais robusto como Winston ou Morgan seria benéfico para produção.
- **Documentação da API:** A API não tem documentação automática (Swagger/OpenAPI). Isto tornaria mais fácil para outros desenvolvedores entenderem e usar a API.
- **Rate Limiting:** Não está implementada proteção contra rate limiting, o que deixa a API vulnerável a ataques de force brute ou DDoS.
- **CORS Aberto:** As configurações CORS atuais são permissivas (origin: true). Em produção, deve ser mais restritivo.

4.1.2 Base de Dados

- **Falta de Índices:** Não foram criados índices adicionais para otimizar consultas frequentes, o que pode afetar a performance com um grande volume de dados.
- **Migrations:** O esquema da base de dados é gerido manualmente. Um sistema de migrations como TypeORM, Prisma ou Drizzle ORM seria útil para gerir alterações ao esquema.
- **Backup e Recovery:** Não está implementado nenhum sistema de backup automático ou estratégia de recovery.
- **Connection Pooling:** As configurações de connection pooling não estão otimizadas para produção.
- **CSRF Protection:** Não está implementada proteção contra CSRF, o que é importante para aplicações com sessões baseadas em cookies.
- **CSP Headers:** Não estão configurados Content Security Policy headers para proteger contra XSS.
- **Security Headers:** Faltam headers de segurança como X-XSS-Protection, X-Content-Type-Options, etc.
- **Input Sanitization:** Embora as queries SQL utilizem prepared statements, não há sanitização adicional de inputs para prevenir outros tipos de ataques.
- **Password Complexity:** Não há validação de complexidade de passwords (mínimo de caracteres, caracteres especiais, etc.).

4.2 Refactoring Necessário

4.2.1 Organização de Código

- **Estrutura de Diretórios:** A estrutura atual do frontend (scripts/) poderia ser mais bem organizada, possivelmente seguindo um padrão como MVC ou Feature-based.
- **Separação de Concerns:** Algumas funcionalidades estão misturadas. Por exemplo, os services às vezes contêm lógica de apresentação que deveria estar nos components.
- **Duplicate Code:** Existe algum código duplicado, especialmente na validação de inputs em diferentes rotas.

4.3 Ideias para Desenvolvimento Futuro

4.3.1 Novas Funcionalidades

- **Sistema de Classificação:** Permitir que os utilizadores classifiquem conteúdos, atualizando a classificação média.
- **Comentários:** Adicionar um sistema de comentários aos conteúdos.
- **Listas de Reprodução:** Permitir que os utilizadores criem e partilhem listas de reprodução personalizadas.
- **Pesquisa Avançada:** Implementar pesquisa com filtros avançados (ano, classificação, etc.).
- **Notificações:** Sistema de notificações para novos conteúdos ou atividades dos perfis.
- **Partilha Social:** Permitir partilhar conteúdos em redes sociais.
- **Subscrições:** Implementar um sistema de subscrição de categorias para receber notificações de novos conteúdos.

4.3.2 Melhorias Técnicas

- **Implementar Testes:** Adicionar testes unitários e de integração para frontend e backend (exemplo: Playwright).
- **CI/CD Pipeline:** Configurar um pipeline de CI/CD para automatizar testes e deployments (exemplo: GitHub Actions ou Woodpecker CI).
- **Dockerização:** Criar containers Docker ou Pods (K8s) para facilitar o deployment e garantir consistência entre ambientes.
- **TypeScript:** Migrar o código JavaScript para TypeScript para melhorar a tipagem e deteção de erros.
- **Monitorização:** Adicionar monitorização da aplicação (metrics, logging centralizado, Grafana, etc.).
- **API Versioning:** Implementar versionamento da API para permitir alterações sem breaking changes.

4.3.3 Melhorias de UX/UI

- **Animações:** Adicionar animações suaves para transições entre páginas e interações.
- **Offline Support:** Implementar cache do frontend para permitir alguma funcionalidade offline.
- **PWA:** Transformar a aplicação numa Progressive Web App para melhor integração com dispositivos móveis.
- **Multi-idioma:** Adicionar suporte para múltiplos idiomas

4.3.4 Expandir o Sistema

- **Integração com APIs Externas:** Conectar a APIs como TMDB para obter informações de filmes automaticamente.
- **Upload de Imagens:** Permitir que os administradores façam upload de imagens para os conteúdos em vez de usar URLs externas.
- **Sistema de Tags:** Adicionar um sistema de tags mais flexível para classificar conteúdos.
- **Recomendações Colaborativas:** Implementar um sistema de recomendações baseadas em utilizadores semelhantes.
- **Estatísticas:** Dashboard de estatísticas para administradores (conteúdos mais vistos, utilizadores ativos, etc.).

4.4 Conclusão

Embora o sistema ESTFlix atual implemente todas as funcionalidades requisitadas e funcione corretamente, existem várias áreas onde melhorias poderiam ser feitas. Estas limitações e sugestões de desenvolvimento futuro servem como ponto de partida para quem possa vir a continuar o desenvolvimento do projeto.

A priorização destas melhorias deve ter em conta:

1. Questões de segurança (CSRF, CSP, etc.)
2. Melhorias de performance (caching, pagination)
3. Testes automatizados
4. Nova funcionalidades para utilizadores

5. Esquema da Base de Dados

Neste apêndice apresentamos o esquema completo da base de dados MySQL do sistema ESTFlix, incluindo todas as tabelas, chaves primárias e estrangeiras, e restrições.

Está localizada na pasta /database/schema.sql.

6. Open Source

O código está acessível num repositório público no GitHub (<https://github.com/diogowho/estflix>) com uma licença Open Source permissiva.